



UNIWERSYTET VIZJA

KIERUNEK: INFORMATYKA

Maciej Mowel

studia niestacjonarne

nr albumu 42935

Aplikacja PWA: Notatnik StudyQuest

Dokumentacja do projektu dyplomowego

przygotowana pod kierunkiem

dra inż. Marcina Kacprowicza

Warszawa, 2026

1. Podstawowe informacje

Nazwa projektu	<i>Aplikacja PWA: StudyQuest</i>
Cel projektu	Celem projektu jest stworzenie aplikacji PWA, pomagającej nauczycielom i uczniom na każdym poziomie nauki w ich codziennych przygotowaniach do zajęć. Aplikacja ma za zadanie pomóc w nauce oraz działać jako personalny asystent uczniów bądź nauczycieli.
Krótki opis projektu	Utworzona aplikacja PWA jest platformą rozwijającą podstawowe działanie aplikacji typu Notatnik, aplikacja ta będzie poprawiała efektywność uczenia się poprzez integrację nowoczesnych technik zapamiętywania z mechanizmami grywalizacji. Kluczowym wyróżnikiem projektu jest system rozwoju postaci typu RPG (Role-Playing Game), w którym użytkownik zdobywa punkty doświadczenia oraz kolejne poziomy za aktywność związaną z przyswajaniem wiedzy. Głównym źródłem progresu jest system automatycznego generowania quizów: dzięki wykorzystaniu AI, użytkownik może jednym przyciskiem przekształcić swoje notatki w testy wyboru ABCD, co pozwala na natychmiastową weryfikację wiedzy. Dodatkowo, platforma oferuje rozbudowane moduły planu zajęć oraz kalendarza, zaprojektowane z myślą o organizacji pracy zarówno uczniów, jak i nauczycieli. Całość tworzy kompleksowe środowisko wspierające systematyczność i samodyscyplinę, zamieniając rutynowe sporządzanie notatek w interaktywne i angażujące doświadczenie edukacyjne.
Analiza konkurencji	Rynek aplikacji mobilnych wspierających proces edukacji i notowania jest nasycony, jednak większość dostępnych rozwiązań skupia się na jednej, wybranej funkcjonalności, rzadko kiedy możemy spotkać aplikacje, które skupiają się na przyciągnięciu uwagi osób młodszych. 1. Systemy zarządzania wiedzą (Notion, Obsidian) Notion jest obecnie liderem w segmencie wszechstronnych notatników. Platforma ta oferuje potężne narzędzia do organizacji treści oraz moduł Notion AI, który potrafi streszczać teksty czy poprawiać stylistykę. • Słabe strony: Mimo ogromnych możliwości, Notion jest narzędziem skomplikowanym (wysoki próg wejścia) i brakuje mu natywnych mechanizmów grywalizacji, takich jak systemy XP czy poziomy postaci. Użytkownik musi samodzielnie budować struktury kalendarza, co dla przeciętnego ucznia może być zbyt czasochłonne.

	2. Platformy do nauki z wykorzystaniem fiszek (Quizlet, Anki) Quizlet oraz Anki to najpopularniejsze narzędzia wykorzystujące metodę aktywnego przypominania. Quizlet wprowadził elementy grywalizacji oraz funkcję „Q-Chat” • Słabe strony: Narzędzia te służą głównie do powtarzania gotowych zestawów danych, a nie do tworzenia rozbudowanych notatek. Proces przejścia z własnej notatki do testu często wymaga ręcznego kopiowania treści. Ponadto grywalizacja w tych aplikacjach jest powierzchowna i nie oferuje rozwoju stałego awatara użytkownika w modelu RPG. 3. Aplikacje do grywalizacji życia (Habitica) Habitica to najpopularniejszy przykład zamiany codziennych obowiązków w grę RPG. Użytkownik zdobywa doświadczenie i ekwipunek za realizację zadań. • Słabe strony: Habitica jest doskonałym menedżerem zadań, ale całkowicie brakuje jej funkcji notatnika oraz integracji z AI, która mogłaby automatycznie generować testy z treści naukowych. Nie jest to narzędzie dedykowane bezpośrednio do nauki, lecz do budowania nawyków. Wnioski: Analiza wykazuje istotną lukę rynkową. Istnieje zapotrzebowanie na narzędzie, które „odciąży” ucznia w procesie przygotowywania materiałów do powtórek i jednocześnie utrzyma jego motywację na wysokim poziomie poprzez mechanizmy znane z gier wideo. Projektowana aplikacja łączy te światy, stanowiąc kompleksowe centrum dowodzenia dla współczesnego studenta lub nauczyciela.
Wykaz zastosowanych technologii	<i>Język programowania: JavaScript</i> <i>Framework serwerowy: Express</i> <i>Backend: Node.js</i> <i>Baza Danych: Supabase (API)</i> Integracja AI: Gemini AI API Środowisko programistyczne: Visual Studio Code Tester aplikacji Android studio code Hosting: Render

Opis stosu technologicznego i uzasadnienie wybranych technologii	Opis powiązań pomiędzy elementami stosu Aplikacja łączy się pośrednio poprzez API z chmurową bazą danych Supabase. Dane o notatkach i postępach RPG są przechowywane w formacie JSON, co zapewnia pełną spójność formatu danych między bazą a aplikacją. Proces generowania testów realizowany jest poprzez asynchroniczne zapytania do Gemini API. Tekst notatki jest przesyłany do modelu, który zwraca ustrukturyzowane pytania testowe. Całość procesu projektowego została poprzedzona przygotowaniem makiety i w narzędziu Figma, co pozwoliło na precyzyjne zaprogramowanie warstwy wizualnej w Visual Studio Code i bieżącą weryfikację efektów na emulatorze w Android Studio. Uzasadnienie 1. Aplikacja Wybór PWA podyktowany był pomysłem stworzenia aplikacji na telefonu jak i na komputery. 2. MongoDB: ZastosowanieMongoDB Atlas jest uzasadnione specyfiką notatek i systemu RPG. Notatki mogą mieć różną długość i strukturę, a profil gracza może ewoluować w trakcie rozwoju projektu. MongoDB pozwala na elastyczne zmienianie struktury danych bez konieczności definiowania sztywnych relacji, co przyspiesza iteracje w projekcie. 3. Google Gemini AI: Wybór Gemini API wynika z jego wysokiej wydajności w przetwarzaniu języka naturalnego oraz dużego okna kontekstowego, co pozwala na analizę nawet bardzo długich notatek studenta. Jest to rozwiązanie kosztowo efektywne dla studenta, a jednocześnie oferuje najnowocześniejsze możliwości generowania logicznych pytań sprawdzających wiedzę. 4. Zestaw VSC : Visual Studio Code jest lekkim i wydajnym edytorem, który dzięki licznym wtyczkom idealnie wspiera Node.js i express. Android Studio dostarcza niezbędnych narzędzi do debugowania niskopoziomowego oraz emulacji różnych urządzeń mobilnych, co gwarantuje poprawność działania aplikacji na różnych wielkościach ekranów.
---	---

2. Kluczowe zagadnienia związane z realizacją projektu

1. Architektura aplikacji i organizacja kodu w środowisku React

StudyQuest opiera się na architekturze 3-tier architecture, czyli Klient-Serwer-Baza danych. Głównym założeniem projektowym było stworzenie modularnego kodu, który jest łatwy w utrzymaniu oraz jest gotowy na możliwy “upgrade”.

Aplikacja została podzielona na logiczne moduły funkcjonalne:

- **Komponenty prezentacyjne (UI):** Odpowiedzialne wyłącznie za wyświetlanie danych/ekranów (np. Notatki, Kalendarz, Plan zajęć, Mój Profil).
- **Komponenty kontenerowe:** Zarządzające logiką biznesową i stanem aplikacji. Przykładowo, moduł QuizGenerator odpowiada za pobranie treści notatki, komunikację z API oraz obsługę błędów.
- **Konteksty (Context API):** Wykorzystane do zarządzania stanem globalnym, co jest kluczowe w przypadku systemu RPG. Stan gracza (poziom, punkty XP, ekwipunek) musi być dostępny w całej aplikacji – zarówno w widoku profilu, jak i w widoku notatnika czy kalendarza.
- Ze względu na charakter PWA, kluczowe są pliki `manifest.json` oraz `Service Worker`. Dzięki temu aplikacja jest poprawnie indeksowana przez przeglądarki mobilne jako aplikacja instalowalna, umożliwiając dodanie skrótu do ekranu głównego oraz pracę w trybie offline.

2. Implementacja Bazy danych

Warstwa danych została zrealizowana w oparciu o nierelacyjną bazę danych Supabase. Wybór technologii podyktowany był potrzebą elastyczności w zarządzaniu strukturą danych oraz wysoką wydajnością, niezbędną dla płynnego działania aplikacji internetowej.

Kluczowe aspekty implementacji obejmują:

- **Model dokumentowy:** Zamiast tradycyjnych tabel, dane przechowywane są w elastycznych dokumentach (format BSON/JSON). Zapewnia to naturalną integrację z warstwą wizualną napisaną w JavaScript, eliminując konieczność skomplikowanego mapowania obiektowo-relacyjnego. Pozwala to na łatwą rozbudowę obiektów.

- **Warstwa pośrednia API:** Aplikacja PWA łączy się z bazą danych bezpośrednio poprzez dedykowane **API**. Aplikacja kliencka wysyła asynchroniczne żądania HTTP, które są odbierane przez serwer, a następnie tłumaczone na zapytania do Supabase.
- **Optymalizacja dla PWA:** Wymiana danych odbywa się w formacie JSON. Taka architektura pozwala na szybkie odświeżanie widoków bez konieczności przeładowywania całej strony.

3. Integracja z modelem językowym Gemini AI

Najbardziej zaawansowanym elementem technicznym projektu jest moduł generowania quizów, który jest możliwy przy użyciu API od Gemini AI

Proces przetwarzania notatki wygląda następująco:

- **Konstrukcja zapytania (Prompt):** Do API wysyłane jest żądanie zawierające nie tylko treść notatki, ale również specjalny prompt do uzyskania testu. Instrukcja ta narzuca modelowi AI ścisły format odpowiedzi – w tym przypadku format JSON.
- **Parsowanie odpowiedzi:** Odpowiedź z Gemini API jest ciągiem znaków. Aplikacja implementuje mechanizm try-catch do parsowania tego ciągu na obiekt JavaScript, który następnie jest renderowany jako interaktywny formularz.
- **Obsługa błędów:** W przypadku, gdy model AI zwróci odpowiedź w błędnym formacie lub usługa jest niedostępna, aplikacja wyświetla stosowny komunikat.

4. Implementacja mechaniki RPG i algorytmów progresji

- Kluczowym elementem mającym wzbudzić “apetyt na wiedzę” u użytkownika jest dodatek grywalizacji do aplikacji,
- Zaimplementowano algorytm przyrostu poziomu trudności: Z każdym następnym poziomem ilość potrzebnego XP zwiększa się o 500xp. Gdzie BaseXP jest stałą wartością 100xp. Po awansie uruchamiana jest animacja awansu oraz odblokowywane są nowe elementy interfejsu (nowa ikonka przy poziomie gracza, w ekranie: Mój Profil).

5. Interfejs użytkownika i UX

- Jako aplikacja typu PWA, *StudyQuest* musi działać płynnie na szerokiej gamie urządzeń – od smartfonów z małymi ekranami, po tablety i laptopy. W realizacji warstwy wizualnej wykorzystano podejście *Mobile First*.
- Stylowanie zostało zrealizowane przy użyciu nowoczesnych standardów CSS. Zamiast sztywnych jednostek pikselowych, zastosowano jednostki relatywne (%), co zapewnia skalowalność elementów.
Kluczowe wyzwania UX, które rozwiązano w projekcie:
- W aplikacji zostały utworzone ekrany, każdy z nich jest odpowiedzialny za swoją własną kategorię tzn. Ekran Notatki składa się z utworzonych notatek oraz przycisku z możliwością dodania nowej notatki. Ekran dodania notatki składa się z formularza: temat notatki, wybór przedmiotu dla którego jest tworzona notatka, oraz treść notatki, czas utworzenia notatki przypisywany jest automatycznie. Ekran Planu zajęć: Ekran ten składa się z listy przedmiotów jakie mamy na dzisiaj, przycisku dzięki któremu możemy sobie sami ustawić okres trwania przedmiotu, oraz przerwy pomiędzy zajęciami. Ekran kalendarza: składa się z typowego kalendarza, mamy przycisku w którym znajduje się formularz: lista przedmiotu do wyboru, wybór dnia oraz krótki opis. Ekran logowania, formularz z logowaniem lub rejestracją, Ekran Quizu, Quiz ABCD na podstawie notatki, I ostatni ekran Mój Profil: zawiera on informacje o poziomie użytkownika oraz historii z 3 ostatnimi notatkami.
- **Tryb ciemny (Dark Mode):** Zaimplementowano dynamiczne przełączanie motywów w oparciu o preferencje systemowe użytkownika (`prefers-color-scheme`).

3. Zrzuty ekranu, wizualizacje,

1. Ekran logowania i rejestracji.

✦

Zaloguj się do StudyQuest

E-MAIL

student@uczelnia.pl

HASŁO

Zaloguj się

Nie masz konta? [Zarejestruj się](#)

✦

Dołącz do StudyQuest

NAZWA UŻYTKOWNIKA

Jan Kowalski

E-MAIL

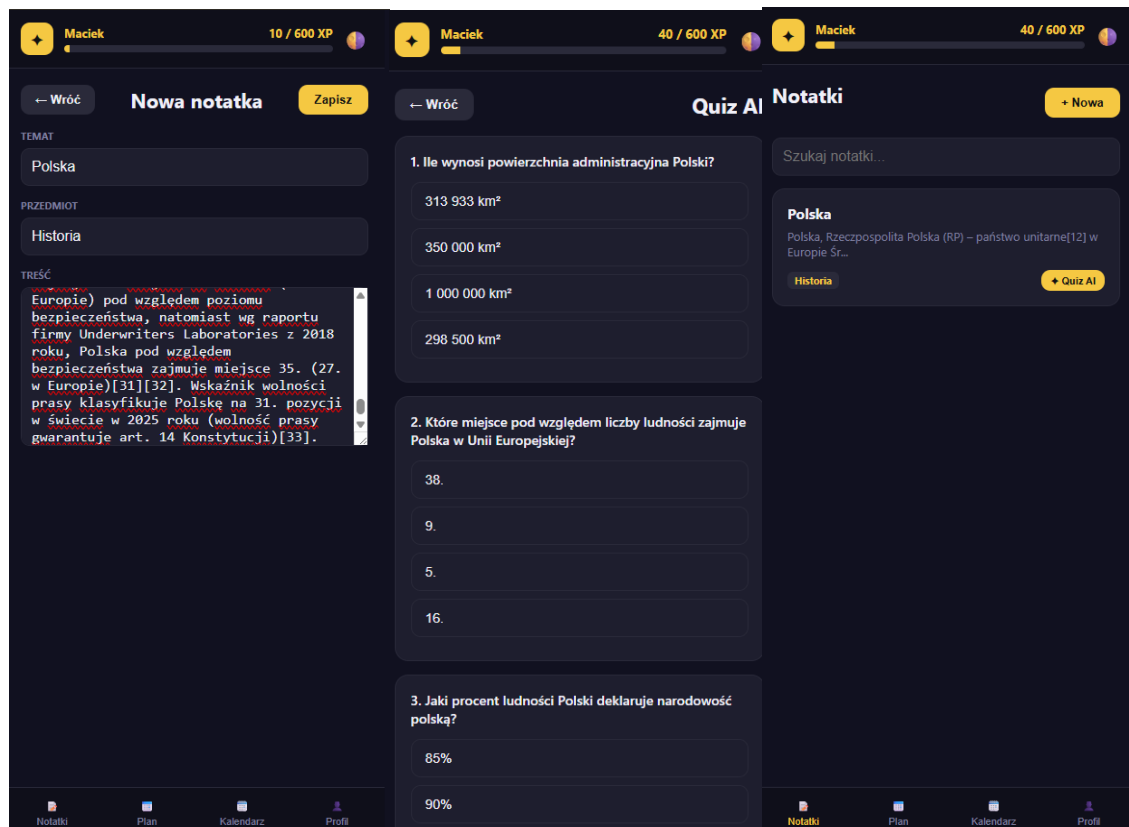
student@uczelnia.pl

HASŁO

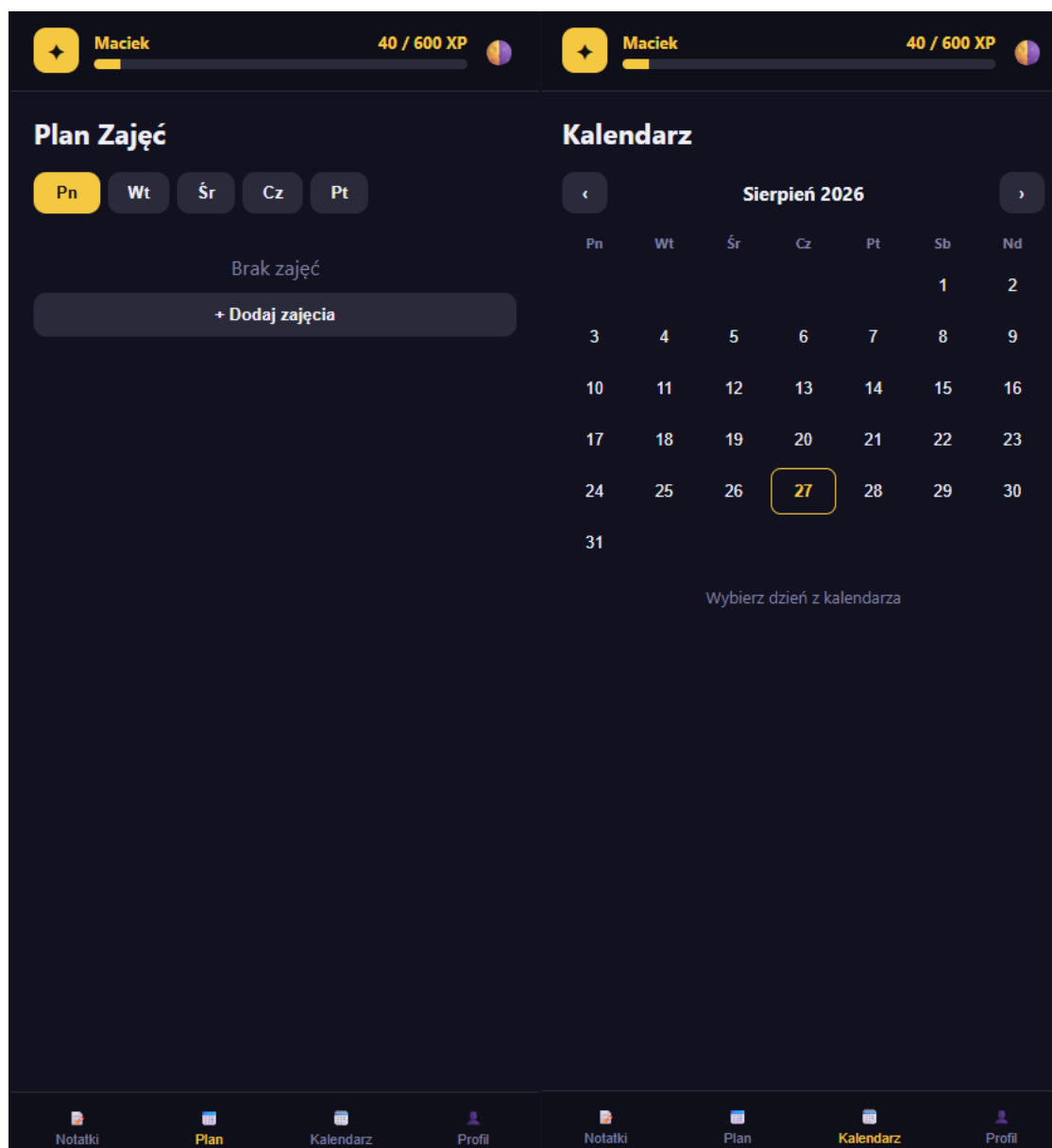
Stwórz konto

Masz już konto? [Zaloguj się](#)

2. Ekran notatki



3. Ekran planu i kalendarza



4. Ekran użytkownika



Maciek

40 / 600 XP



Mój Profil

Maciek

Poziom 2

1

Rozwiązane quizy

40%

Średnia skuteczność

80

XP z quizów

1

Notatki

OSTATNIE QUIZY

Arsenal

27.08.2026 • +80 XP

4/10



Eksport (JSON)



Import (JSON)

Wyloguj się

Resetuj wszystkie dane



Notatki



Plan



Kalendarz



Profil

4. Wnioski i perspektywy rozwoju

1. Wnioski z realizacji projektu

- **Skuteczne wypełnienie luki rynkowej:** Połączenie tradycyjnego notatnika z automatyczną weryfikacją wiedzy (AI) oraz elementami grywalizacji (RPG) stanowi odpowiedź na brak rozwiązań łączących kompleksowość z wysokim poziomem zaangażowania uczniów.
- **Efektywność architektury PWA:** Wybór technologii PWA umożliwił stworzenie wieloplatformowego narzędzia o wysokiej wydajności, gotowego do działania na urządzeniach mobilnych i desktopowych bez potrzeby budowania osobnych aplikacji natywnych.
- **Automatyzacja nauki dzięki Gemini AI:** Integracja z modelem Gemini AI pozwala na błyskawiczne przekształcanie notatek w quizy ABCD, co znacząco skraca czas potrzebny na przygotowanie materiałów powtórkowych.
- **Wzrost motywacji poprzez grywalizację:** Wdrożenie prostego systemu progresji XP i poziomów postaci skutecznie stymuluje systematyczność oraz zmienia rutynowe nauczanie w angażujący proces.

2. Możliwości rozwoju i dalsze perspektywy

- **Rozbudowa mechanik RPG i personalizacja:** Wprowadzenie awatarów użytkownika, drzewka umiejętności, ekwipunku zdobywanego za XP oraz wyzwań dziennych i tygodniowych (Questy).
- **Wielomodalne tworzenie notatek (Multimodal AI):** Wykorzystanie analizy obrazu w Gemini API do automatycznego skanowania notatek ręcznych, zdjęć z tablicy lub plików PDF i bezpośredniego generowania z nich quizów.
- **Różnorodność formatów sprawdzania wiedzy:** Rozszerzenie modułu AI o generowanie pytań otwartych, fiszek (flashcards), zadań typu prawda/fałsz.
- **Moduł społecznościowy i rywalizacja:** Dodanie rankingów znajomych, możliwość udostępniania gotowych notatek i quizów wewnątrz platformy oraz tryb wspólnego rozwiązywania testów.
- **Powiadomienia PUSH i integracje:** Dodanie natywnych powiadomień Push o nadchodzących zajęciach/kolokwiach oraz integracja z zewnętrznymi kalendarzami (Google Calendar, Outlook).